

PropAI — Real Estate Platform

SERVER-SIDE PRODUCT REQUIREMENTS DOCUMENT

Version 1.0 | Backend Architecture, API Design, Database, and AI Integration

1. Server Overview

The PropAI backend is a RESTful API server built with Node.js, Express, and TypeScript. It powers all data operations, authentication, real-time communication, AI feature orchestration, and background job processing for the PropAI real estate platform. The server is designed for scalability, reliability, and production-readiness from the ground up.

Core Responsibilities:

- Authentication & authorization (JWT + role-based access control).
- CRUD operations for all entities: users, properties, inquiries, messages, blogs, reviews, notifications.
- AI API orchestration: Claude/Gemini calls for all 4 AI features.
- RAG (Retrieval-Augmented Generation) pipeline for the AI chatbot.
- Real-time communication via Socket.io.
- Background job processing via BullMQ.
- Rate limiting, logging, caching, and error tracking.

2. Backend Technology Stack

Technology	Purpose	Version/Notes
Node.js	JavaScript runtime environment	v20 LTS
Express.js	Web framework for REST API	v4+
TypeScript	Type safety throughout backend codebase	v5+, strict mode
Prisma ORM	Database access layer with type-safe queries	v5+
PostgreSQL	Primary relational database	v15+

Technology	Purpose	Version/Notes
Redis	Caching + BullMQ queue backend + session store	v7+
Socket.io	Real-time bidirectional communication	v4+
BullMQ	Background job queue system	v4+
Winston	Structured logging library	v3+
express-rate-limit	HTTP request rate limiting	Latest
Sentry	Error tracking and monitoring	@sentry/node v7+
Better Auth	Not used directly in Express, but shares DB	
multer storage cloudinary		
multer	File/image upload handling	v1+
Cloudinary / S3	Image storage CDN	Cloudinary SDK
Nodemailer	Email sending (verification, reset)	v6+
Zod	Server-side request validation	v3+
cors	Cross-origin resource sharing	v2+
helmet	HTTP security headers	v7+
compression	Gzip response compression	v1+
dotenv	Environment variable management	v16+

3. Project Structure

The server follows a modular, feature-based architecture. Each feature module contains its own routes, controller, service, and types.

```

server/
├── src/
│   ├── config/           # App config, env validation, constants
│   ├── middleware/       # Auth, error handler, rate limiter, logger
│   ├── modules/
│   │   ├── auth/         # login, register, refresh, reset-password
│   │   ├── users/        # user CRUD, profile, role management
│   │   ├── properties/   # property CRUD, search, filters
│   │   ├── inquiries/    # buyer-seller inquiry threads
│   │   └── messages/     # real-time message storage

```

```

├── reviews/      # property reviews and ratings
├── blogs/        # blog CRUD
├── notifications/ # notification management
├── analytics/    # chart data endpoints
├── reports/      # report management (admin)
├── ai/           # all AI feature endpoints
├── services/
│   ├── ai.service.ts      # Claude/Gemini API calls
│   ├── rag.service.ts     # RAG pipeline for chatbot
│   ├── cache.service.ts   # Redis caching helpers
│   ├── email.service.ts   # Nodemailer email sending
│   ├── upload.service.ts  # Cloudinary image uploads
│   └── queue.service.ts   # BullMQ job definitions
├── socket/
│   ├── socket.server.ts   # Socket.io initialization
│   ├── chat.handler.ts    # Message events
│   └── notification.handler.ts
├── jobs/
│   ├── email.job.ts       # Email sending job processor
│   └── ai-recommendation.job.ts
├── prisma/
│   └── schema.prisma      # Full database schema
├── utils/                # Helpers, formatters, validators
├── types/                # Shared TypeScript types/interfaces
├── app.ts                # Express app setup
├── index.ts              # Entry point – starts HTTP + Socket server
├── prisma/
│   ├── schema.prisma
│   ├── seed.ts
│   └── migrations/
├── .env                  # Environment variables (never committed)
├── .env.example          # Template with all required variables
└── tsconfig.json

```

4. Environment Variables

All environment variables must be defined and validated at startup using a config module (e.g., using Zod to parse process.env). The application must fail fast if required variables are missing.

Variable	Description	Example
PORT	Server listening port	5000

Variable	Description	Example
NODE_ENV	Environment mode	development / production
DATABASE_URL	PostgreSQL connection string (Prisma)	postgresql://user:pass@host/db
REDIS_URL	Redis connection string	redis://localhost:6379
BETTER_AUTH_SECRET	Must match Next.js BETTER_AUTH_SECRET exactly	long-secret-string
NEXT_PUBLIC_APP_URL	Next.js app URL for CORS & cookie validation	http://localhost:3000
ANTHROPIC_API_KEY	Claude API key for AI features	sk-ant-...
GOOGLE_CLIENT_ID	Google OAuth client ID (NextAuth)	xxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET	Google OAuth client secret	GOCSPX-...
CLOUDINARY_CLOUD_NAME	Cloudinary cloud name	propai-cloud
CLOUDINARY_API_KEY	Cloudinary API key	123456789
CLOUDINARY_API_SECRET	Cloudinary API secret	secret_string
SMTP_HOST	Email server host	smtp.gmail.com
SMTP_PORT	Email server port	587
SMTP_USER	Email address for sending	noreply@propai.com
SMTP_PASS	Email password or app password	app-specific-password
CLIENT_URL	Frontend URL for CORS and email links	http://localhost:3000
SENTRY_DSN	Sentry project DSN for error tracking	https://xxx@sentry.io/yyy

5. Database Schema (Prisma)

5.1 User Model (Better auth)

Better Auth models (auto-managed):

- `user` — id, name, email, emailVerified, image, role, phone, status, createdAt, updatedAt
- `session` — id, expiresAt, token, userId
- `account` — id, userId, type, provider, providerAccountId, refresh_token, access_token, ...
- `verification` — id, identifier, value, expiresAt, type

```
enum Role { BUYER SELLER ADMIN }
enum UserStatus { ACTIVE BANNED SUSPENDED }
```

Application models (our own):

- `UserProfile` — id (foreign key to `user.id`), bio, preferredCities, preferredTypes, priceRangeMin, priceRangeMax, avatar, agencyName, ...
- All other models (`Property`, `SavedProperty`, `Inquiry`, `Conversation`, `Message`, `Review`, `Blog`, etc.) remain exactly as in v1.0, only now referencing `userId` that maps to `user.id`.

5.2 Property Model

```
model Property {
  id          String      @id @default(cuid())
  title       String
  slug        String      @unique
  description  String
  price       Float
  priceType   PriceType @default(TOTAL) // TOTAL | PER_MONTH
  status      PropertyStatus @default(PENDING)
  listingType ListingType  // FOR_SALE | FOR_RENT
  type        PropertyType // APARTMENT | HOUSE | VILLA | LAND |
  COMMERCIAL | OFFICE
  city        String
  area        String
  address     String
  latitude    Float?
  longitude   Float?
  bedrooms    Int?
  bathrooms   Int?
  areaSqft    Float?
  floorNumber Int?
  totalFloors Int?
}
```

```

parkingSpaces Int?
yearBuilt      Int?
furnished      FurnishedStatus @default(UNFURNISHED)
amenities      String[] // array: 'wifi', 'pool', 'gym', etc.
images         String[] // array of Cloudinary URLs
tags           String[]
viewCount      Int    @default(0)
avgRating      Float  @default(0)
reviewCount    Int    @default(0)
sellerId       String
seller         User    @relation(fields: [sellerId], references: [id])
createdAt      DateTime @default(now())
updatedAt      DateTime @updatedAt

savedBy        SavedProperty[]
inquiries      Inquiry[]
reviews        Review[]
browsingHistory BrowsingHistory[]
reports        Report[]
}

```

```

enum PropertyStatus { PENDING ACTIVE INACTIVE REJECTED }
enum ListingType    { FOR_SALE FOR_RENT }
enum PropertyType    { APARTMENT HOUSE VILLA LAND COMMERCIAL OFFICE }
enum FurnishedStatus { FURNISHED SEMI_FURNISHED UNFURNISHED }

```

5.3 Other Models

Model	Key Fields	Purpose
SavedProperty	id, userId, propertyId, savedAt	Buyer bookmarks — unique constraint on (userId, propertyId)
Inquiry	id, message, status (PENDING/REPLIED/CLOSED), buyerId, sellerId, propertyId, createdAt	Initial contact message from buyer to seller about a property
Conversation	id, buyerId, sellerId, propertyId, lastMessageAt	Message thread between buyer and seller — one per buyer-seller-property combination
Message	id, content, senderId, conversationId, read (bool), createdAt	Individual message in a conversation

Model	Key Fields	Purpose
Review	id, rating (1-5), comment, userId, propertyId, createdAt	Buyer review on a property — unique constraint on (userId, propertyId)
Blog	id, title, slug, content, coverImage, tags[], status (DRAFT/PUBLISHED), views, authorId, createdAt	Blog post entity
Notification	id, type (enum), title, message, read (bool), userId, relatedEntityId, relatedEntityType, createdAt	In-app notification per user
BrowsingHistory	id, userId, propertyId, viewedAt	Tracks which properties a buyer viewed — used for AI recommendations
Report	id, reason, type (PROPERTY/USER), reportedEntityId, reporterId, status (PENDING/REVIEWED), createdAt	User-submitted reports for admin review
PasswordReset	id, token (hashed), userId, expiresAt, used (bool)	One-time tokens for password reset flow
NewsletterSubscriber	id, email, subscribedAt, active (bool)	Email subscribers list

6. API Routes — Complete Specification

6.1 Auth Routes — /api/auth

All `/api/auth/*` routes are removed from Express. Better Auth in Next.js now handles:

- POST `/api/auth/sign-up/email`
- POST `/api/auth/sign-in/email`
- POST `/api/auth/sign-in/social`
- GET `/api/auth/callback/google`
- POST `/api/auth/sign-out`
- GET `/api/auth/get-session`
- POST `/api/auth/forget-password`
- POST `/api/auth/reset-password`
- GET `/api/auth/verify-email`

6.2 User Routes — /api/users

(No change in routes, but the `req.user` object now comes from the Better Auth session.)

Method	Endpoint	Auth	Description
GET	/api/users/profile	Auth required	Get the current user's full profile.
PATCH	/api/users/profile	Auth required	Update: name, phone, bio, avatar, preferredCities, preferredTypes, priceRange.
PATCH	/api/users/change-password	Auth required	Body: { currentPassword, newPassword }. Validates current password before updating.
DELETE	/api/users/account	Auth required	Soft-delete user account. Body: { password } for confirmation.
GET	/api/users	Admin only	List all users. Query: ?role=BUYER&status=ACTIVE&search=john&page=1&limit=15.
GET	/api/users/:id	Admin only	Get any user's profile.
PATCH	/api/users/:id/status	Admin only	Ban or unban a user. Body: { status, reason? }.
PATCH	/api/users/:id/role	Admin only	Change user role. Body: { role }.
DELETE	/api/users/:id	Admin only	Hard delete user and all their data.

6.3 Property Routes — /api/properties

Method	Endpoint	Auth	Description
GET	/api/properties	Public	List properties with filtering. Query: ?type=APARTMENT&city=Dhaka&listingType=FOR_RENT&minPrice=5000&maxPrice=30000&bedrooms=2&rating=4&search=gulshan&sortBy=price_asc&page=1&limit=12
GET	/api/properties/:id	Public	Get single property by ID. Increments viewCount by 1. Returns full property with seller info, avg rating.
GET	/api/properties/slug/:slug	Public	Get property by URL slug for SEO-friendly URLs.
POST	/api/properties	Seller/Admin	Create new property. Body: full property object. Status auto-set to PENDING.
PATCH	/api/properties/:id	Seller (own) / Admin	Update property fields. Seller can only update their own.

Method	Endpoint	Auth	Description
DELETE	/api/properties/:id	Seller (own) / Admin	Delete property and its images from Cloudinary.
PATCH	/api/properties/:id/status	Admin only	Approve, reject, or deactivate. Body: { status, rejectionReason? }.
GET	/api/properties/seller/my-listings	Seller only	Get all properties belonging to current seller. Query: ?status=ACTIVE&page=1&limit=10.
POST	/api/properties/:id/save	Buyer only	Toggle save/unsave property. Returns { saved: true/false }.
GET	/api/properties/saved	Buyer only	Get all saved properties for current buyer.
POST	/api/properties/:id/view	Auth required	Record view in BrowsingHistory for AI recommendations.

6.4 Inquiry Routes — /api/inquiries

Method	Endpoint	Auth	Description
POST	/api/inquiries	Buyer only	Send inquiry to seller about a property. Body: { propertyId, message }. Creates Inquiry + Conversation if not exists.
GET	/api/inquiries/sent	Buyer only	Get all inquiries sent by buyer. Query: ?status=PENDING&page=1.
GET	/api/inquiries/received	Seller only	Get all inquiries received by seller. Query: ?status=PENDING&page=1.
PATCH	/api/inquiries/:id/status	Seller only	Update inquiry status. Body: { status: REPLIED CLOSED }.
GET	/api/inquiries/admin	Admin only	All inquiries with user info. Full filter support.

6.5 Message Routes — /api/messages

Method	Endpoint	Auth	Description
GET	/api/messages/conversations	Auth required	Get all conversations for current user. Returns list with last message and unread count.
GET	/api/messages/conversations/:conversationId	Auth required	Get last 50 messages in a conversation. User must be participant. Query:

Method	Endpoint	Auth	Description
			?before={messageId}&limit=50 for pagination.
POST	/api/messages/conversations/:conversationId/read	Auth required	Mark all messages in conversation as read. Used when user opens conversation.

Note: Actual message sending happens via Socket.io, not REST. Messages are persisted by the Socket.io handler.

6.6 Review Routes — /api/reviews

Method	Endpoint	Auth	Description
GET	/api/reviews/property/:propertyId	Public	Get all reviews for a property. Returns reviews + rating breakdown (count per star level).
POST	/api/reviews/property/:propertyId	Buyer only	Submit a review. Body: { rating, comment }. One review per user per property. Recalculates property avgRating.
PATCH	/api/reviews/:id	Owner only	Edit own review. Body: { rating, comment }.
DELETE	/api/reviews/:id	Owner or Admin	Delete review. Recalculates property avgRating.

6.7 Blog Routes — /api/blogs

Method	Endpoint	Auth	Description
GET	/api/blogs	Public	List published blogs. Query: ?search=&tag=&page=1&limit=9.
GET	/api/blogs/:slug	Public	Get blog by slug. Increments view count.
POST	/api/blogs	Admin only	Create blog. Body: { title, content, coverImage, tags, status }.
PATCH	/api/blogs/:id	Admin only	Update blog content or status.
DELETE	/api/blogs/:id	Admin only	Delete blog.
GET	/api/blogs/admin/all	Admin only	All blogs including drafts. Filter by status.

6.8 Notification Routes — /api/notifications

Method	Endpoint	Auth	Description
GET	/api/notifications	Auth required	Get notifications for current user. Query: ?read=false&page=1&limit=20.
PATCH	/api/notifications/:id/read	Auth required	Mark single notification as read.
POST	/api/notifications/read-all	Auth required	Mark all notifications as read.
DELETE	/api/notifications/:id	Auth required	Delete a notification.

6.9 Analytics Routes — /api/analytics

Method	Endpoint	Auth	Description
GET	/api/analytics/seller/overview	Seller only	Seller stats: total listings, total views, total inquiries, profile views.
GET	/api/analytics/seller/views-over-time	Seller only	Daily view counts per listing. Query: ?propertyId=&days=30.
GET	/api/analytics/seller/inquiries-per-listing	Seller only	Inquiry count grouped by listing.
GET	/api/analytics/seller/type-distribution	Seller only	Seller's own listings grouped by property type.
GET	/api/analytics/admin/overview	Admin only	Platform stats: total users, listings, pending, inquiries, this-week signups.
GET	/api/analytics/admin/users-over-time	Admin only	Daily new user registrations. Query: ?days=30.
GET	/api/analytics/admin/listings-per-city	Admin only	Active listings grouped by city.
GET	/api/analytics/admin/type-distribution	Admin only	All active listings grouped by type.
GET	/api/analytics/admin/inquiries-per-day	Admin only	Daily inquiry volume. Query: ?days=30.

6.10 Report Routes — /api/reports

Method	Endpoint	Auth	Description
POST	/api/reports	Auth required	Submit a report. Body: { type: PROPERTY USER, reportedEntityId, reason }.
GET	/api/reports	Admin only	All reports. Query: ?type=&status=PENDING&page=1&limit=15.
PATCH	/api/reports/:id	Admin only	Update report status. Body: { status: REVIEWED }.
DELETE	/api/reports/:id	Admin only	Dismiss report.

6.11 Upload Routes — /api/upload

Method	Endpoint	Auth	Description
POST	/api/upload/image	Auth required	Upload single image to Cloudinary. Returns { url, publicId }. Max 5MB. Types: jpg, png, webp.
POST	/api/upload/images	Auth required	Upload multiple images (max 15). Returns array of { url, publicId }.
DELETE	/api/upload/image/:publicId	Auth required	Delete image from Cloudinary by publicId.

6.12 Newsletter Routes — /api/newsletter

Method	Endpoint	Auth	Description
POST	/api/newsletter/subscribe	Public	Body: { email }. Adds to subscriber list. Sends welcome email via BullMQ.
POST	/api/newsletter/unsubscribe	Public	Body: { email, token }. Token from unsubscribe link in emails.
GET	/api/newsletter/subscribers	Admin only	List all subscribers with pagination.

7. AI Routes — /api/ai (Full Specification)

All AI endpoints call Anthropic Claude (claude-sonnet-4-20250514) via the Anthropic Node.js SDK. All endpoints require authentication. All responses are structured JSON. All endpoints apply strict rate limiting: 10 requests per minute per user.

7.1 AI Content Generator — /api/ai/generate

Method	Endpoint	Auth	Description
POST	/api/ai/generate/property-description	Seller only	Generate property description. See spec below.
POST	/api/ai/generate/blog-post	Admin only	Generate full blog post content. See spec below.
POST	/api/ai/generate/property-tags	Seller only	Auto-generate property tags from title + description.

7.1.1 POST /api/ai/generate/property-description

Request Body:

```
{
  type: string,           // 'APARTMENT' | 'HOUSE' | 'VILLA' | etc.
  city: string,           // 'Dhaka'
  area: string,           // 'Gulshan'
  price: number,          // 25000
  priceType: string,      // 'PER_MONTH' | 'TOTAL'
  bedrooms: number,       // 3
  bathrooms: number,      // 2
  areaSqft: number,       // 1400
  amenities: string[],    // ['wifi', 'elevator', 'security']
  listingType: string     // 'FOR_RENT' | 'FOR_SALE'
}
```

System Prompt sent to Claude:

```
You are an expert real estate copywriter specializing in Bangladeshi
property market.

Generate a professional, persuasive property listing. Return ONLY valid
JSON, no markdown.
```

User Prompt constructed from request body data.

Expected JSON Response:

```
{
  description: string,     // 200-300 word professional description
  highlights: string[],    // 5 bullet point key features
  metaDescription: string  // 150 char SEO meta description
}
```

7.1.2 POST /api/ai/generate/blog-post

Request Body: { title: string, keywords: string[] (optional, max 5) }

Expected JSON Response:

```
{
  introduction: string, // opening paragraph 100 words
  sections: [
    { heading: string, content: string } // 3-4 sections, 150 words each
  ],
  conclusion: string, // 80 word conclusion
  tags: string[] // 5 SEO tags
}
```

7.2 AI Smart Recommendations — /api/ai/recommendations

7.2.1 POST /api/ai/recommendations

Auth: Buyer only.

How the endpoint works:

- Step 1: Fetch user's browsing history (last 15 viewed property IDs) from BrowsingHistory table.
- Step 2: Fetch user's saved property IDs.
- Step 3: Fetch user's profile preferences (preferredCities, preferredTypes, priceRange).
- Step 4: Fetch 60 most recent ACTIVE property IDs from database (candidates).
- Step 5: Build prompt for Claude with user context + candidate property summaries.
- Step 6: Claude returns 8 recommended property IDs with reasoning.
- Step 7: Fetch full property data for those IDs. Return to client.

Prompt Construction:

```
User Profile: Prefers {types} in {cities}, budget {min}-{max} BDT.
Recently viewed: {title, type, city, price for each viewed property}.
Saved properties: {title, type, city for each saved}.
Available properties to recommend from: {id, title, type, city, price,
bedrooms, amenities}.
Return ONLY JSON: { recommendations: [ { id: string, reason: string } ] } -
8 items.
```

Request Body: {} (server fetches everything from DB using JWT user ID)

Response:

```
{
  recommendations: [
    {
      property: PropertyObject, // full property with seller info
    }
  ]
}
```

```

        reason: string // AI-generated reason: 'Matches your
preference for...'
    }
]
}

```

Caching: Cache result per user in Redis for 30 minutes. Key: ai:recommendations:{userId}. Invalidate on new browse or save event.

7.3 AI Data Analyzer — /api/ai/analyze

7.3.1 POST /api/ai/analyze/seller

Auth: Seller only.

Request Body:

```

{
  viewsOverTime: { date: string, views: number }[],
  inquiriesPerListing: { title: string, count: number }[],
  typeDistribution: { type: string, count: number }[]
}

```

Expected JSON Response:

```

{
  insights: [
    {
      title: string, // short headline for insight
      insight: string, // 1-2 sentence detailed insight
      type: 'positive' | 'warning' | 'neutral'
    }
  ] // 4 items
}

```

7.3.2 POST /api/ai/analyze/admin

Auth: Admin only.

Same structure as seller but with platform-wide data: usersOverTime, listingsPerCity, typeDistribution, inquiriesPerDay.

7.4 RAG Chatbot — /api/ai/chat

7.4.1 POST /api/ai/chat/message

Auth: Public (but user context added if logged in for personalization).

Request Body:

```

{
  message: string,           // user's question
  conversationHistory: [     // last 6 messages for context
    { role: 'user' | 'assistant', content: string }
  ]
}

```

RAG Pipeline Steps on the server:

- Step 1 — Intent Extraction: Parse user message to extract: location mentions, property type, price range, number of bedrooms, listing type (sale/rent).
- Step 2 — Database Retrieval: Query PostgreSQL with extracted filters. Retrieve up to 8 matching ACTIVE properties. Fields: id, title, city, area, type, price, bedrooms, bathrooms, areaSqft, amenities, listingType.
- Step 3 — Context Building: Format retrieved properties into a structured string for the AI context.
- Step 4 — AI Call: Send to Claude with system prompt: 'You are PropAI's helpful real estate assistant. Answer ONLY based on the provided property data. If no matching properties exist in the data, say so clearly. Do not invent properties. Be friendly and specific.'
- Step 5 — Response: Return AI's natural language response to client.

Response:

```

{
  reply: string,           // AI's natural language response
  properties: [...]       // array of property IDs mentioned - frontend fetches full data
}

```

Rate Limiting: 20 messages per 10 minutes per IP/user to prevent AI API cost abuse.

8. Middleware Stack

8.1 Global Middleware (applied to all routes)

Middleware	Configuration
helmet()	Sets 11 security headers. Prevents XSS, clickjacking, MIME sniffing.
cors()	Origin: CLIENT_URL env variable. Credentials: true. Methods: GET,POST,PUT,PATCH,DELETE. Headers: Content-Type, Authorization.
compression()	Gzip all responses. Threshold: 1024 bytes.
express.json()	Body parser. Limit: 2mb.
express.urlencoded()	URL-encoded body parser. Extended: true.

Middleware	Configuration
morgan/Winston logger	Log every request: method, URL, status code, response time, user ID (from JWT if present).

8.2 Authentication Middleware

8.3 Rate Limiting Middleware

Limiter	Window	Max Requests	Applied To
General API limiter	15 minutes	200 requests	All /api/* routes
Auth limiter	15 minutes	10 requests	/api/auth/login, /api/auth/register
AI limiter	1 minute	10 requests	All /api/ai/* routes
AI Chat limiter	10 minutes	20 requests	/api/ai/chat/*
Upload limiter	1 hour	30 requests	/api/upload/*
Contact form limiter	1 hour	5 requests	/api/contact

- All rate limiters return 429 Too Many Requests with Retry-After header.
- Response body: { error: 'Too many requests', retryAfter: number }.

8.4 Global Error Handler

Single error-handling middleware at the end of the Express middleware chain:

- Catches all errors passed via next(error).
- Distinguishes between operational errors (known, safe to expose) and programming errors (unknown).
- For operational errors (ZodError, Prisma errors, custom AppError): returns structured JSON with status code and message.
- For unknown errors: returns 500 with generic message. Full error sent to Sentry.
- Never exposes stack traces in production.

Response format for all errors:

```
{
  success: false,
  error: {
    code: string,      // e.g. 'VALIDATION_ERROR', 'NOT_FOUND',
'UNAUTHORIZED'
    message: string,  // human-readable message
    fields?: { [field: string]: string } // only for validation errors
  }
}
```

```
}  
}
```

9. Logging System (Winston)

Winston is configured with multiple transports for different environments.

9.1 Log Levels

Level	Usage
error	Unhandled exceptions, database connection failures, AI API failures, critical errors.
warn	Rate limit hits, failed authentication attempts, deprecated API usage.
info	Server start, every HTTP request (method + route + status + response time), AI feature usage.
debug	Detailed SQL queries (dev only), Socket.io events, cache hits/misses.

9.2 Transport Configuration

- Console transport: All levels in development. Colorized, readable format.
- File transport (combined.log): All info+ logs. JSON format. Max size: 20MB. Max files: 5 (rotating).
- File transport (error.log): Errors only. JSON format.
- Sentry transport: error level only. Sends to Sentry with full context.

9.3 HTTP Request Logger

Every request logged with:

```
{  
  timestamp: ISO string,  
  method: 'GET' | 'POST' | ...,  
  url: '/api/properties?city=Dhaka',  
  statusCode: 200,  
  responseTime: '45ms',  
  userId: 'clxxx...' | null,  
  userRole: 'BUYER' | null,  
  ip: '192.168.1.1'  
}
```

10. Caching Strategy (Redis)

Cache Key Pattern	TTL	What is Cached	Invalidation Trigger
properties:explore:{queryHash}	5 minutes	Property listing results for a specific filter/sort/page combination. queryHash is MD5 of query params.	New property created, property status updated, property deleted.
properties:featured	10 minutes	8 featured properties shown on homepage.	Any property status changes.
properties:detail:{propertyId}	10 minutes	Full property detail object including seller info and avg rating.	Property updated, new review added.
ai:recommendations:{userId}	30 minutes	AI recommendations result per user.	User saves/unsaves a property, user views 5 new properties.
analytics:admin:overview	5 minutes	Admin dashboard stat cards.	New user registered, new listing created.
analytics:seller:overview:{userId}	5 minutes	Seller's own stat cards.	Seller's listing gets new view or inquiry.
blogs:list:{queryHash}	15 minutes	Blog listing page results.	Blog published, updated, or deleted.
blogs:detail:{slug}	30 minutes	Full blog post content.	Blog updated.

10.1 Cache Implementation Pattern

```
// Generic cache-aside pattern
async function getCachedOrFetch<T>(key: string, ttl: number, fetcher: () => Promise<T>): Promise<T> {
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);
  const data = await fetcher();
  await redis.setex(key, ttl, JSON.stringify(data));
  return data;
}
```

11. Real-Time System (Socket.io)

11.1 Socket Server Setup

- Socket.io server attached to the same HTTP server as Express.
- CORS: same origin as CLIENT_URL.
- Authentication: Socket.io middleware verifies JWT token from socket.handshake.auth.token on connect.
- Connection refused with 401 if token invalid or missing.

11.2 Socket Rooms

Room Name Pattern	Who Joins	Purpose
user:{userId}	Each user on connect	Private room for receiving personal notifications
conversation:{conversationId}	Both participants when they open a conversation	Real-time message delivery in a specific conversation

11.3 Socket Events

Event Name	Direction	Payload	Action
connection	Client → Server	JWT token in handshake.auth	Authenticate, join user:{userId} room.
join:conversation	Client → Server	{ conversationId: string }	Validate user is participant, join conversation room.
message:send	Client → Server	{ conversationId, content: string }	Save message to DB, emit message:receive to conversation room.
message:receive	Server → Client	{ id, content, senderId, senderName, senderAvatar, conversationId, createdAt }	New message received — client appends to chat.
user:typing	Client → Server	{ conversationId, isTyping: boolean }	Emit typing:status to other participant in room.
typing:status	Server → Client	{ userId, isTyping: boolean }	Show/hide typing indicator in chat.
notification:new	Server → Client	{ id, type, title, message, relatedEntityId }	Push real-time notification to user:{userId} room.
notification:read	Client → Server	{ notificationId }	Mark notification as read in DB.

Event Name	Direction	Payload	Action
disconnect	Auto	-	User leaves all rooms automatically.

11.4 Notification Trigger Points

Event	Who Gets Notified	Notification Type
New inquiry submitted	Seller	NEW_INQUIRY
Seller replies to inquiry	Buyer	INQUIRY_REPLIED
New message in conversation	Recipient	NEW_MESSAGE
Listing approved by admin	Seller	LISTING_APPROVED
Listing rejected by admin	Seller	LISTING_REJECTED
User account banned	User	ACCOUNT_BANNED
New property matches buyer preference	Buyer	NEW_MATCH

12. Background Job System (BullMQ)

12.1 Queue Setup

- BullMQ uses Redis as the queue backend (same Redis instance).
- Queue defined in `src/services/queue.service.ts`.
- Worker processes defined in `src/jobs/*.ts`.
- Workers run in the same process as the server in development. In production, can be extracted to separate worker process.

12.2 Job Definitions

Queue Name	Job Name	Trigger	What it Does
email-queue	send-verification-email	New user registration	Send email with verification link via Nodemailer.
email-queue	send-password-reset-email	Forgot password request	Send email with password reset link.
email-queue	send-inquiry-notification-email	New inquiry received	Email seller: 'You have a new

Queue Name	Job Name	Trigger	What it Does
			inquiry for {property}'.
email-queue	send-newsletter-welcome	Newsletter subscription	Welcome email to new subscriber.
email-queue	send-listing-approved-email	Admin approves listing	Email seller: 'Your listing has been approved'.
notification-queue	create-notification	Any trigger event	Create Notification record in DB + emit Socket.io event to user.
ai-queue	precompute-recommendations	User browses 5th new property	Pre-run AI recommendations in background, cache result. So buyer dashboard loads instantly.

12.3 Job Error Handling

- All jobs have a retry policy: 3 attempts with exponential backoff (1s, 5s, 30s).
- After 3 failures: job moved to dead letter queue, error logged to Winston + Sentry.
- Email job failures are silent to user — never show email failure in API response.

13. Error Tracking (Sentry)

- Sentry initialized at app startup with SENTRY_DSN from environment.
- Sentry Express integration captures: all unhandled exceptions, Express route errors, slow transactions (> 2s).
- User context attached when available: { id, email, role } from req.user.
- Custom tags added: { environment, feature } — e.g., feature: 'ai-recommendations'.
- Sensitive data scrubbing: passwords, tokens, API keys never sent to Sentry. Configured via denyUrls and before-send hook.
- Performance monitoring: tracesSampleRate: 0.1 (10% of transactions) in production to avoid cost.

14. Authentication & Security System

14.1 JWT Strategy

Token	Expiry	Storage	Rotation
Access Token	15 minutes	Client memory / Authorization header	Refreshed automatically when expired using refresh token
Refresh Token	7 days	httpOnly, Secure, SameSite=Strict cookie	Rotated on each use (refresh token rotation). Old token invalidated.

14.2 Password Security

- Passwords hashed with bcrypt, salt rounds: 12.
- Password reset tokens: crypto.randomBytes(32) hashed with SHA256 before DB storage.
- Reset token expiry: 1 hour.
- Minimum password requirements enforced: 8 chars, 1 uppercase, 1 lowercase, 1 number.

14.3 Security Headers (Helmet)

- Content-Security-Policy: strict, allowing only own origin and Cloudinary for images.
- X-Frame-Options: DENY (prevents clickjacking).
- X-Content-Type-Options: nosniff (prevents MIME type sniffing).
- Referrer-Policy: strict-origin-when-cross-origin.
- HSTS: max-age 1 year, includeSubDomains (production only).

14.4 Input Validation

- All request bodies validated with Zod schemas before reaching controller.
- Validation middleware wraps all route handlers.
- Unknown fields are stripped (strict Zod schemas). No extra data passes through.
- SQL injection: impossible with Prisma parameterized queries.
- File upload validation: MIME type + extension must match. Max file size: 5MB per image.

15. API Response Standards

15.1 Success Response Format

```

{
  success: true,
  data: T,                // the response payload
  message?: string,       // optional human-readable message
  pagination?: {          // only for list endpoints
    page: number,
    limit: number,
    total: number,
    totalPages: number,
    hasNext: boolean,
    hasPrev: boolean
  }
}

```

15.2 HTTP Status Code Usage

Code	Meaning	When Used
200 OK	Success	GET, PATCH, DELETE operations succeed
201 Created	Resource created	POST (register, create property, create blog)
400 Bad Request	Client error	Validation error, malformed request, business rule violation
401 Unauthorized	Not authenticated	Missing/invalid/expired token
403 Forbidden	Not authorized	Valid token but wrong role
404 Not Found	Resource missing	Property, user, blog not found
409 Conflict	Duplicate resource	Email already registered, already reviewed this property
422 Unprocessable	Semantic error	Valid format but invalid logic (e.g., start date > end date)
429 Too Many Requests	Rate limited	Rate limiter triggered
500 Internal Server Error	Server fault	Unhandled exceptions — generic message only

16. Deployment & Production Setup

16.1 Server Deployment (Render / Railway)

Configuration	Value
Platform	Render.com (recommended) or Railway.app
Instance Type	Render: Standard (\$7/month). Railway: Starter plan.
Build Command	npm install && npx prisma generate && npm run build
Start Command	node dist/index.js
Health Check Endpoint	GET /api/health — returns { status: 'ok', timestamp, uptime }
Auto Deploy	Connected to GitHub main branch — auto deploy on push
Environment Variables	All .env variables configured in Render/Railway dashboard — never committed to git

16.2 Database (Supabase / Neon / Railway PostgreSQL)

- Use Supabase free tier (500MB) or Neon for serverless PostgreSQL.
- DATABASE_URL must be the direct connection string (not pooler for Prisma migrations).
- For production, use the pooler URL for regular queries (better connection management).
- Run migrations: npx prisma migrate deploy — never prisma migrate dev in production.
- Seed demo data: npx prisma db seed — creates demo users, 20+ sample properties, reviews, blogs.

16.3 Redis (Upstash)

- Use Upstash Redis free tier — serverless Redis with REST API.
- REDIS_URL from Upstash dashboard.
- Compatible with ioredis (used by BullMQ and caching).

16.4 Image Storage (Cloudinary)

- Free tier: 25GB storage, 25GB bandwidth/month — sufficient for demo.
- Create a dedicated 'propai' folder in Cloudinary.
- Set upload preset to 'signed' for security.
- Image transformation on upload: auto quality, auto format, max width 1920px.

17. Database Seed Data Requirements

The seed script (prisma/seed.ts) must create enough data for a convincing demo. No empty pages or tables.

Entity	Quantity	Details
Admin User	1	Email: admin@propai.com, Password: Admin@123. Role: ADMIN.
Seller/Agent Users	3	demo@seller.com (Password: Seller@123) + 2 others. Verified, active.
Buyer Users	3	demo@buyer.com (Password: Buyer@123) + 2 others. With browsing history.
Properties	30+	Mixed types (Apartment, House, Land, Villa, Commercial). Mix of FOR_SALE and FOR_RENT. Multiple cities. All ACTIVE. Real-looking titles, descriptions, multiple image URLs.
Reviews	20+	Various ratings (3-5 stars) across multiple properties. With comments.
Blogs	6	4 published, 2 drafts. Real titles (e.g., 'Top 10 Areas to Buy Property in Dhaka 2025'). Cover images. Tags.
Inquiries	10+	Mix of PENDING, REPLIED, CLOSED statuses.
Notifications	15+	Various types per user. Some read, some unread.
Browsing History	20+	Buyer users with varied browsing history for AI recommendations demo.
Newsletter Subscribers	5	Sample email addresses.

18. Backend Engineering Checklist

18.1 Advanced Backend Features (All 5 Required)

Feature	Implemented Via	Where Applied	Status
Rate Limiting	express-rate-limit	All routes — general, auth, AI, upload limiters	Required
Logging	Winston + Morgan	Every request, every error, all AI calls	Required
Caching	Redis via ioredis	Property listings, featured, AI recommendations, analytics	Required
Queue System	BullMQ + Redis	Email sending, notifications, AI precompute jobs	Required
Error Tracking	Sentry @sentry/node	All unhandled errors in production with context	Required

18.2 Security Checklist

- Passwords hashed with bcrypt (rounds: 12)
- JWT with short-lived access tokens (15m) + httpOnly refresh cookie rotation
- All inputs validated with Zod before reaching database
- Helmet security headers on all responses
- Rate limiting on all routes, stricter on auth and AI endpoints
- CORS restricted to CLIENT_URL origin only
- No sensitive data in logs or error responses
- File uploads: type validation + size limits
- SQL injection impossible via Prisma parameterized queries
- Refresh token blacklisting in Redis on logout

18.3 Production Readiness Checklist

- Health check endpoint responding at /api/health
- Graceful shutdown: server handles SIGTERM — closes DB connections, flushes logs
- All environment variables documented in .env.example
- Prisma migrations tracked in git and applied via migrate deploy in CI/CD
- Seed script creates complete demo data
- All API endpoints return consistent response format
- Winston logging to file in production
- Sentry error tracking active in production
- README includes setup instructions, env variable list, and demo credentials

END OF SERVER-SIDE PRD

PropAI — Real Estate Platform | Server PRD v1.0